Regular Expressions

Regular expressions (RegEx) in Python are used for searching, matching, and manipulating text based on specific patterns through the built-in `re` module.

(1) Compile()

Returns Module contains compile() Function to compile a Pattern into RegexObject.

pattern = re.compile("ab")

`pattern = re.compile("ab")` :

1. You are telling Python: "I am going to be looking for the sequence '**ab**' a lot."
2. Python takes that string `"ab"` and turns it into a high-performance **Pattern Object**.
3. This object is now stored in the variable `pattern`.

Let's look at a more practical example: **Validating a Username.**
Imagine you want a username to only have **lowercase letters** and **numbers**, and it must be at least **5 characters long**.

```
import re

# 1. Compile the "Rules" for a username
# [a-z0-9] means lowercase letters or numbers
# {5,} means at least 5 characters long
user_rules = re.compile(r"^[a-z0-9]{5,}$")

# 2. Use the compiled rules to check different names
test_names = ["user1", "abc", "JohnDoe", "cool_cat123"]

for name in test_names:
    if user_rules.match(name):
        print(f"✅ '{name}' is a valid username.")
    else:
        print(f"❌ '{name}' is NOT valid.")
```

(2) finditer()

`re.finditer()` method is used to find **all** occurrences of a pattern in a string.

Unlike `findall()`, which returns a simple list of strings, `finditer()` returns an **iterator** that yields **match objects**.

On Match object we can call the following methods.

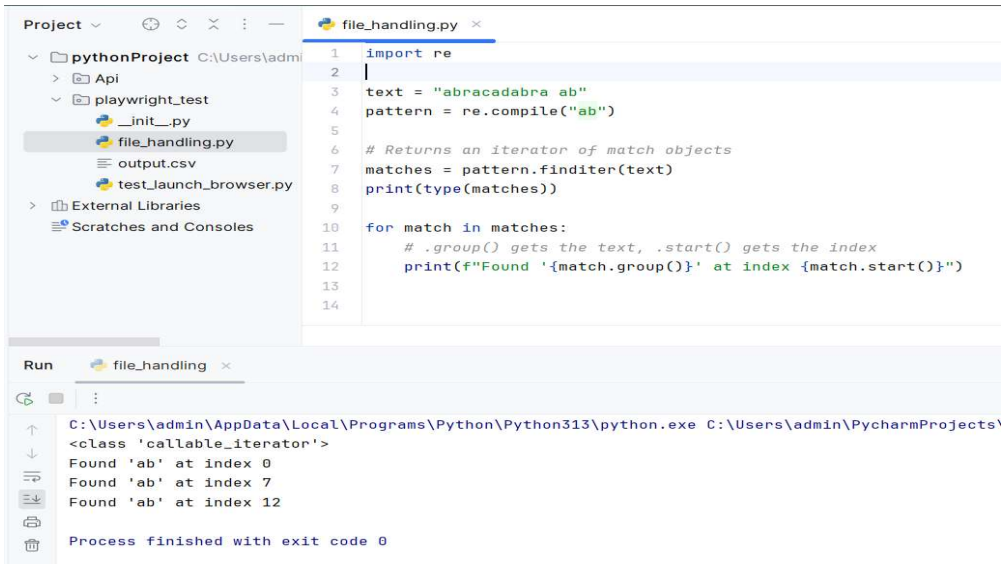
1) **start() ➔ Returns start index of the match**
2) **end() ➔ Returns end+1 index of the match**
3) **group() ➔ Returns the matched string**

```
import re

text = "abracadabra ab"
pattern = re.compile("ab")

# Returns an iterator of match objects
matches = pattern.finditer(text)

for match in matches:
    # .group() gets the text, .start() gets the index
    print(f"Found '{match.group()}' at index {match.start()}")
```



Character Classes:

We can use character classes to search a group of characters

1) [abc] → Either a OR b OR c
2) [^abc] → Except a and b and c
3) [a-z] → Any Lower case alphabet symbol
4) [A-Z] → Any upper case alphabet symbol
5) [a-zA-Z] → Any alphabet symbol
6) [0-9] → Any digit from 0 to 9
7) [a-zA-Z0-9] → Any alphanumeric character
8) [^a-zA-Z0-9]→ Any alphanumeric characters(Special Characters)

```
import re

text = "Hello! User_99@gmail.com"

# 1. Find all uppercase letters
print(re.findall(r"[A-Z]", text))
# Output: ['H', 'U']

# 2. Find all digits
print(re.findall(r"[0-9]", text))
# Output: ['9', '9']

# 3. Find all special characters (not letters or numbers)
print(re.findall(r"[^a-zA-Z0-9]", text))
# Output: ['!', ' ', '_', '@', '.', '.']
```

Project

pythonProject C:\Users\admi
- Api
- playwright_test
  - __init__.py
  - file_handling.py
  - output.csv
  - test_launch_browser.py

file_handling.py

```python
import re
x = "[^abc]"
matcher=re.finditer(x, string: "a7b@k9z")
for match in matcher:
    print(match.start(),".....",match.group())
```

for match in matcher

Run file_handling

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe C:\Users\admin\PycharmProjects\pythonProject\playwrigl
1 ..... 7
3 ..... @
4 ..... k
5 ..... 9
6 ..... z

Process finished with exit code 0
```

Pre defined Character Classes:

1) \s ➔ Space character
2) \S ➔ Any character except space character
3) \d ➔ Any digit from 0 to 9
4) \D ➔ Any character except digit
5) \w ➔ Any word character [a-zA-Z0-9]
6) \W ➔ Any character except word character (Special Characters)
7) . ➔ Any character including special characters

```python
import re
matcher=re.finditer("x","a7b k@9z")
for match in matcher:
    print(match.start(),".....",match.group())
```

x = \s:	x = \S:	x = \d:	x = \D:	x = \w:	x = \W:	x = .
3	0 ..... a	1 7	0 ..... a	0 ..... a	3	0 ..... a
	1 7	6 9	2 ..... b	1 7	5 ..... @	1 7
	2 ..... b		3	2 ..... b		2 ..... b
	4 ..... k		4 ..... k	4 ..... k		3 .....
	5 ..... @		5 ..... @	6 9		4 ..... k
	6 9		7 ..... z	7 ..... z		5 ..... @
	7 ..... z					6 9
						7 ..... z

Qunatifiers:

We can use quantifiers to specify the number of occurrences to match.

1) a ➔ Exactly one 'a'
2) a+ ➔ Atleast one 'a'
3) a* ➔ Any number of a's including zero number
4) a? ➔ Atmost one 'a' ie either zero number or one number
5) a{m} ➔ Exactly m number of a's
6) a{m,n} ➔ Minimum m number of a's and Maximum n number of a's.

```
import re
matcher=re.finditer("x","abaabaaab")
for match in matcher:
    print(match.start(),"......",match.group())
```

x = a:	x = a+:	x = a*:	x = a?:	x = a{3}:	x = a{2,4}:
0 ..... a	0 ..... a	0 ..... a	0 ..... a	5 ..... aaa	2 ..... aa
2 ..... a	2 ..... aa	1	1		5 ..... aaa
3 ..... a	5 ..... aaa	2 ..... aa	2 ..... a		
5 ..... a		4	3 ..... a		
6 ..... a		5 ..... aaa	4		
7 ..... a		8	5 ..... a		
		9	6 ..... a		
			7 ..... a		
			8		
			9		

match():

- ✓ We can use match function to check the given pattern at beginning of target string.
- ✓ If the match is available then we will get Match object, otherwise we will get None.

```
import re

text = "Python is amazing"

# 1. Successful Match (at the start)
result1 = re.match(r"Python", text)
if result1:
    print(f"Matched: {result1.group()}") # Output: Python

# 2. Failed Match (in the middle)
result2 = re.match(r"amazing", text)
if result2 is None:
    print("No match found at the beginning!")
```

fullmatch():

- ✓ We can use fullmatch() function to match a pattern to all of target string. i.e complete string should be matched according to given pattern.
- ✓ If complete string matched then this function returns Match object otherwise it returns None.

```
import re

pattern = r"Python"

# 1. Exact match (Success)
print(re.fullmatch(pattern, "Python"))
# Output: <re.Match object; span=(0, 6), match='Python'>

# 2. Partial match at start (Failure)
print(re.fullmatch(pattern, "Python is fun"))
# Output: None (because of " is fun")

# 3. Extra characters (Failure)
print(re.fullmatch(pattern, " Python"))
# Output: None (because of the leading space)
```

search():

- ✓ We can use search() function to search the given pattern in the target string.
- ✓ If the match is available, then it returns the Match object which represents first occurrence of the match.
- ✓ If the match is not available, then it returns None

```python
import re

text = "Error 404: Page not found at 10:30 AM"

# Pattern to find any 3-digit number
pattern = re.compile(r"\d{3}")

# Search looks through the whole string
result = pattern.search(text)

if result:
    print(f"Found match: '{result.group()}'")
    print(f"Located at index: {result.start()}")
else:
    print("No match found.")
```

^ symbol:

We can use ^ symbol to check whether the given target string starts with our provided pattern or not.

Eg: **res = re.search("^Learn",s)**

If the target string starts with learn then it will return Match object,otherwise returns None.

test.py

```python
import re
s="Learning Python is Very Easy"
res=re.search("^Learn",s)
if res != None:
    print("Target String starts with Learn")
else:
    print("Target String Not starts with Learn")
```

Output: **Target String starts with Learn**

$ symbol:

We can use $ symbol to check whether the given target string ends with our provided pattern or not.

Eg: res = re.search("Easy$",s)

If the target string ends with Easy then it will return Match object,otherwise returns None.

test.py

```
1) import re
2) s="Learning Python is Very Easy"
3) res=re.search("Easy$",s)
4) if res != None:
5)    print("Target String ends with Easy")
6) else:
7)    print("Target String Not ends with Easy")
```

Output: Target String ends with Easy

Write a Regular Expression to represent all 10 Digit Mobile Numbers?

Rules:

1) Every Number should contain exactly 10 Digits

2) The 1st Digit should be 7 OR 8 OR 9

We can use pattern

```
pattern = r"^[789]\d{9}$"
```

Breakdown:

- `^` : Matches the beginning of the string (ensures no extra characters before the number).
- `[789]` : Matches the first digit, which must be a **7, 8, or 9**.
- `\d{9}` : Matches exactly **9 more digits** (0-9).
- `$` : Matches the end of the string (ensures the number isn't longer than 10 digits)

We can also write like below

[7-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]

OR [7-9][0-9]{9}

OR [7-9]\d{9}

```
import re

# 1) Get input from the user
n = input("Enter number: ")

# 2) Use fullmatch to check the pattern (Starts with 7-9, followed by 9
digits)
m = re.fullmatch(r"[7-9]\d{9}", n)

# 3) Check if the match exists
if m:
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

```python
import re

# 1) Get input from the user
n = input("Enter number: ")

# 2) Use fullmatch to check the pattern (Starts with 7-9, followed by 9 digits)
m = re.fullmatch( pattern: r"[7-9]\d{9}", n)

# 3) Check if the match exists
if m:
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe C:\Users\admin\PycharmProjects\pythonProject\pla
Enter number: 7353012578
Valid Mobile Number

Process finished with exit code 0
```

Web Scraping by using Regular Expressions

The process of collecting information from web pages is called web scraping. In web scraping to match our required patterns like mail ids, mobile numbers we can use regular expressions.

In web scraping, Regular Expressions (Regex) are primarily used to extract specific data like phone numbers, emails, or prices from the raw HTML text of a page.

import requests

import re

1. Fetch the website content

url = "https://example-contact-page.com"
response = requests.get(url)
html_content = response.text

# 2. Define the pattern (10 digits starting with 7, 8, or 9)
# Note: \b ensures we match whole words and not numbers inside longer strings
phone_pattern = r"\b[7-9]\d{9}\b"

# 3. Extract all matches
mobile_numbers = re.findall(phone_pattern, html_content)

print("Found mobile numbers:", mobile_numbers)

```
import requests
import re

# 1. Fetch the website content
url = "https://example-contact-page.com"
response = requests.get(url)
html_content = response.text

# 2. Define the pattern (10 digits starting with 7, 8, or 9)
# Note: \b ensures we match whole words and not numbers inside longer strings
phone_pattern = r"\b[7-9]\d{9}\b"

# 3. Extract all matches
mobile_numbers = re.findall(phone_pattern, html_content)

print("Found mobile numbers:", mobile_numbers)
```

Write a Python Program to check whether given Car Registration Number is valid Karnataka State Registration Number OR not?

Project

- pythonProject C:\Users\admi
  - Api
  - playwright_test
    - __init__.py
    - file_handling.py
    - output.csv
    - test_launch_browser.py
  - External Libraries
  - Scratches and Consoles

file_handling.py

```python
import re

# 1) Get input from the user
s = input("Enter Karnataka Vehicle Registration Number: ").upper()

# 2) Regex Breakdown for KA:
# KA      : State code for Karnataka
# \d{2}   : Exactly 2 digits for the RTO office (e.g., 01, 19, 51)
# [A-Z]{1,2} : 1 or 2 series letters (e.g., M, AB, G)
# \d{4}   : Exactly 4 digits for the vehicle number
pattern = r"KA\d{2}[A-Z]{1,2}\d{4}"

m = re.fullmatch(pattern, s)

# 3) Check if the match exists
if m:
    print("Valid Karnataka Vehicle Registration Number")
else:
    print("Invalid Karnataka Vehicle Registration Number")
```

Run file_handling

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe C:\Users\admin\PycharmProjects\pythonProject\playwright_t
Enter Karnataka Vehicle Registration Number: KA01M1234
Valid Karnataka Vehicle Registration Number

Process finished with exit code 0
```